

Git hébergé

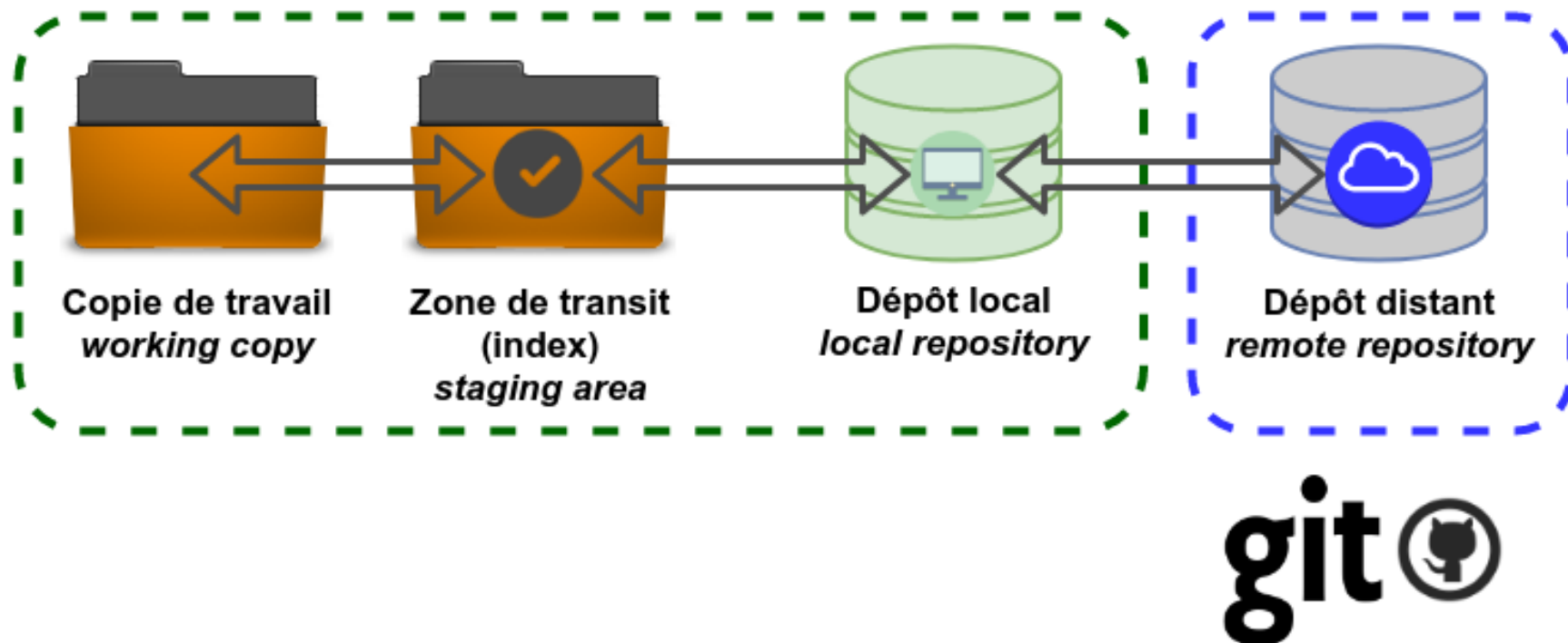
Il est possible d'héberger des projets Git sur un site externe dédié à l'hébergement. Quelques hébergeurs :

- **GitHub** est un service web d'hébergement (lancé en 2008) et de gestion de développement de logiciels, utilisant le logiciel de gestion de versions Git. Site officiel : <https://github.com/>
- **GitLab** est un logiciel libre de forge basé sur Git proposant les fonctionnalités de wiki, un système de suivi des bugs, l'intégration continue et la livraison continue. Site officiel : <https://about.gitlab.com/>
- **Bitbucket Cloud** est un service web d'hébergement et de gestion de développement logiciel utilisant le logiciel de gestion de versions Git. Site officiel : <https://bitbucket.org/>

Liste : <https://git.wiki.kernel.org/index.php/GitHosting>

Notion de dépôt distant

Un **dépôt distant** est un dépôt hébergé sur un serveur, généralement sur Internet.

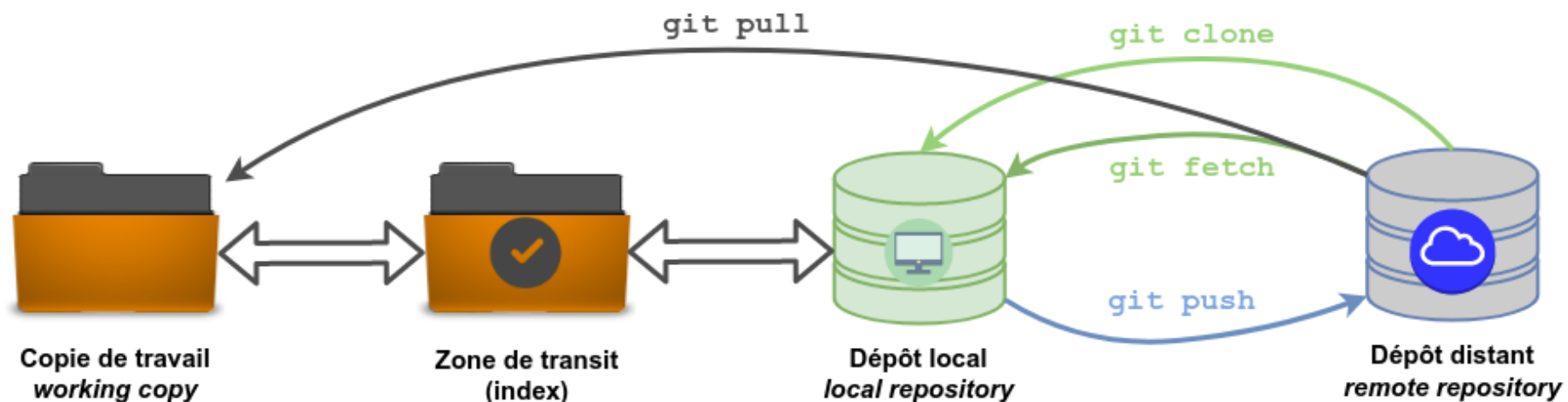


Un dépôt distant peut servir à la sauvegarde et/ou au partage du code d'un projet.

Interagir avec un dépôt distant

Des commandes spécifiques seront utilisées pour synchroniser les dépôts local et distant :

- `git clone` permet d'obtenir une copie d'un dépôt Git existant.
- `git push` publie ("pousse") les nouveaux *commits* du dépôt local sur le dépôt distant.
- `git fetch` récupère l'ensemble des *commits* présents sur le le dépôt distant et met à jour le dépôt local. Elle ne modifie pas le répertoire de travail.
- `git pull` consiste essentiellement en `git fetch` suivi par `git merge`. Le répertoire de travail peut donc être modifié.



Interagir avec GitHub

Il est possible d'interagir avec un dépôt sur GitHub de plusieurs manières :

- L'URL d'accès au dépôt en **SSH** sera de la forme :

`git@github.com:<utilisateur>/<depot>.git`

`https://docs.github.com/en/free-pro-team@latest/github/authenticating-to-github/connecting-to-github-with-ssh`

- L'URL d'accès au dépôt en **HTTPS** sera de la forme :

`https://github.com/<utilisateur>/<depot>.git`

`https:`

`//docs.github.com/en/github/authenticating-to-github/keeping-your-account-and-data-secure/creating-a-personal-access-token`

- La commande `gh` permet l'utilisation de GitHub en la ligne de commande (CLI) : `gh repo clone <utilisateur>/<depot>.git`
`https://cli.github.com/`



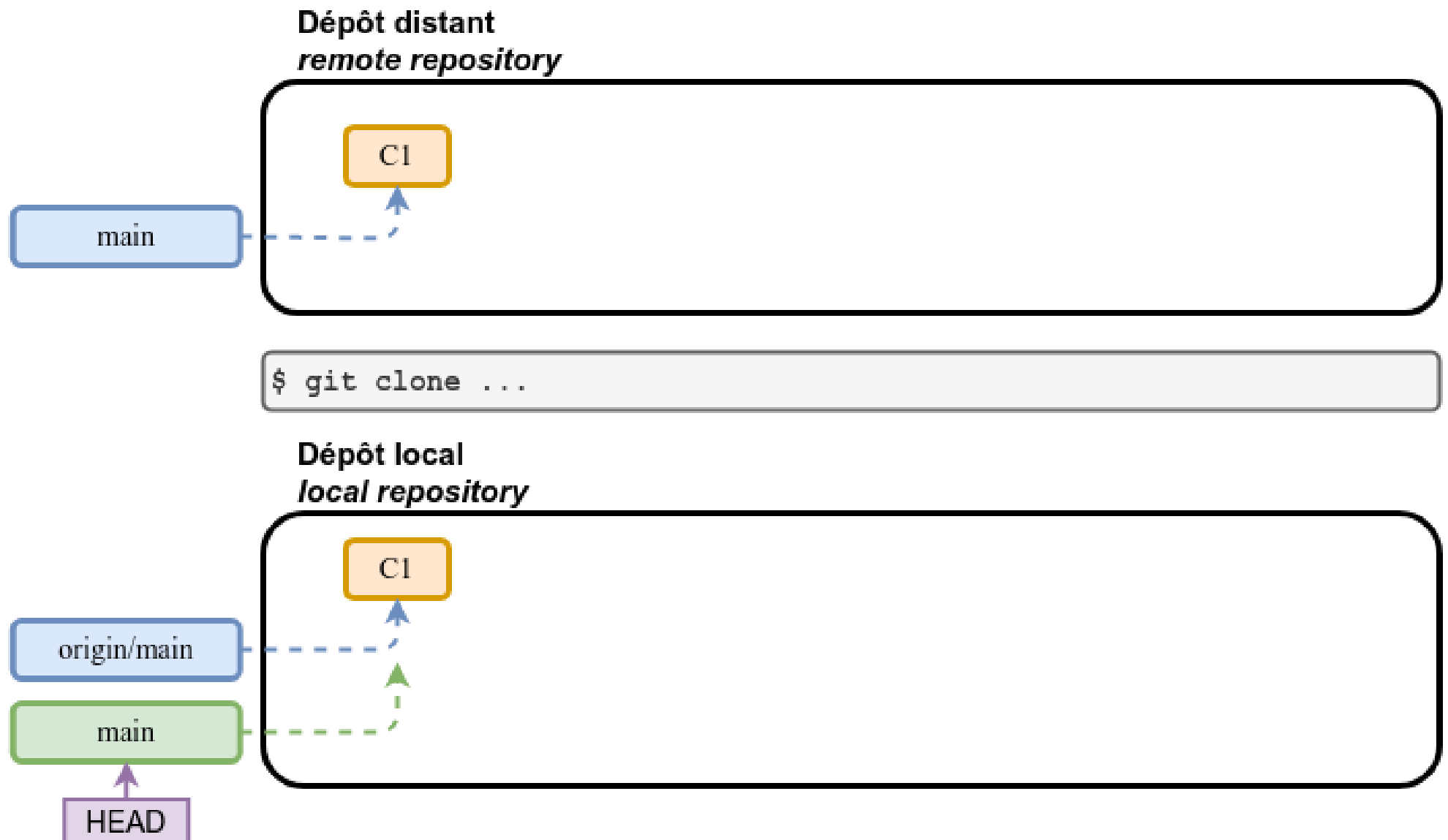
Cloner un dépôt distant

```
$ git clone URL
```

La commande `git clone` effectuera les actions suivantes :

- crée un répertoire du nom du dépôt existant, initialisé avec un répertoire `.git` à l'intérieur,
- nomme automatiquement le serveur distant (*remote*) `origin`,
- tire l'historique,
- crée un pointeur sur l'état actuel de la branche `main` et l'appelle localement `origin/main`
- crée également une branche locale `main` qui démarre au même endroit que la branche `main` distante

Cloner un dépôt distant



Associer un dépôt distant existant

Un dépôt distant (*remote repository*) doit exister sur GitHub par exemple :

```
$ mkdir <depot>
$ cd ./<depot>
$ git init

$ git remote add origin git@github.com:<utilisateur>/<depot>.git
$ git branch -M main
$ git push -u origin main

$ git remote -v
origin git@github.com:<utilisateur>/<depot>.git (fetch)
origin git@github.com:<utilisateur>/<depot>.git (push)
```

Branche de suivi

Une **branche de suivi** (*tracking branch*) est une branche locale qui est en relation directe avec une branche distante (*upstream branch*).

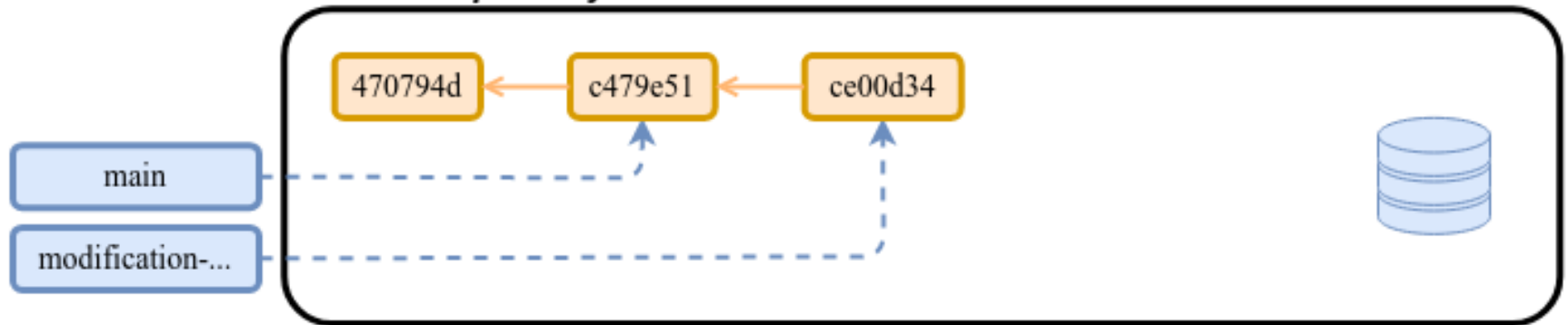
Les branches de suivi peuvent servir :

- à sauvegarder son travail sur la branche dans un dépôt distant
- partager son travail sur la branche avec d'autres développeurs

Voir aussi les options `--track`, `-u` ou `--set-upstream-to`, `--set-upstream` des commandes `checkout`, `branch` et `push`.

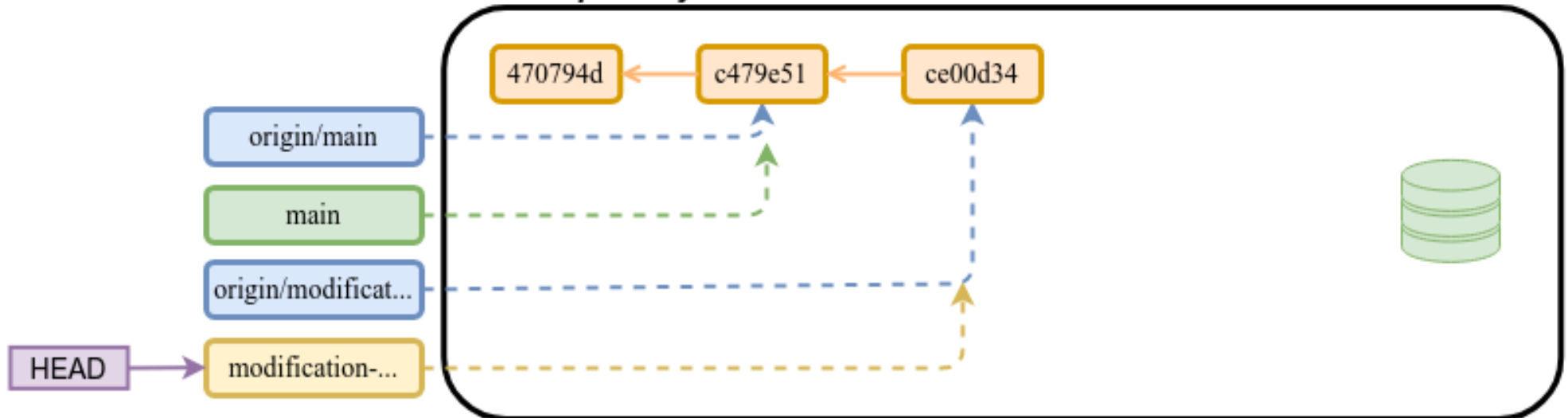
Branche de suivi

Dépôt distant
remote repository



```
$ git push --set-upstream origin modification-fonction
```

Dépôt local
local repository



alle
vignon

Pull Request et Révision de code

Pull Request peut être traduit par « **Proposition de révision** » (PR) : c'est-à-dire une demande de modification ou de contribution.

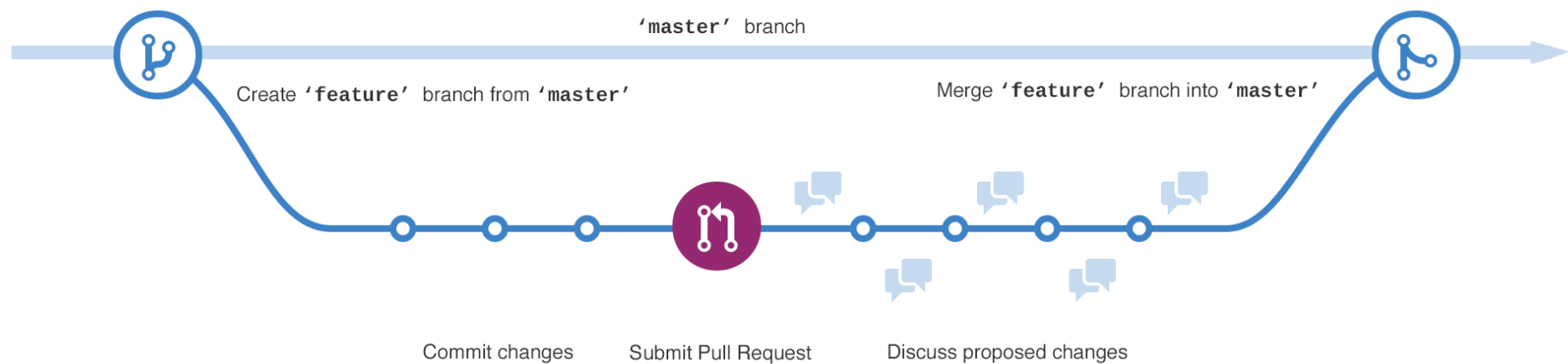
Les *Pull Requests* sont :

- une fonctionnalité facilitant la collaboration des développeurs sur un projet.
- un mécanisme permettant à un développeur d'informer les membres de l'équipe qu'il a terminé un « travail » (une fonctionnalité, une version livrable, un correctif, ...) et de proposer sa contribution au dépôt central.
- une notification aux développeurs pour qu'ils révisent le code puis le fusionnent (*merge*).

Pendant une **révision de code**, les développeurs peuvent discuter de la fonctionnalité (commenter le code, poser des questions, ...) et proposer des adaptations de la fonctionnalité en publiant des *commits* de suivi.



Pull Request

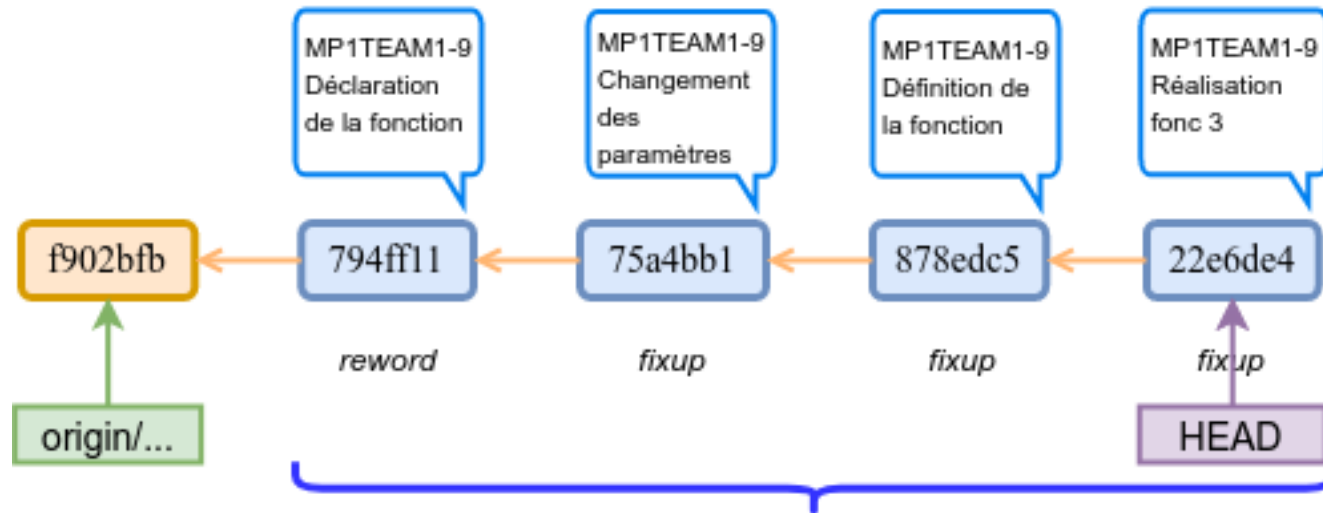


Travail collaboratif

Attention :

- Nettoyer son historique local avant de publier en effectuant un rebasage interactif avec `git rebase -i @upstream`
- Travailler à plusieurs sur une branche de fonctionnalité : faire `git pull --rebase` sur une branche de suivi obsolète
- Supprimer (si besoin) toutes ses modifications et commits locaux et récupérer un dépôt distant « propre » :
`git fetch origin && git reset --hard origin/main`

Rebasage interactif



```
git rebase -i @{upstream}
```

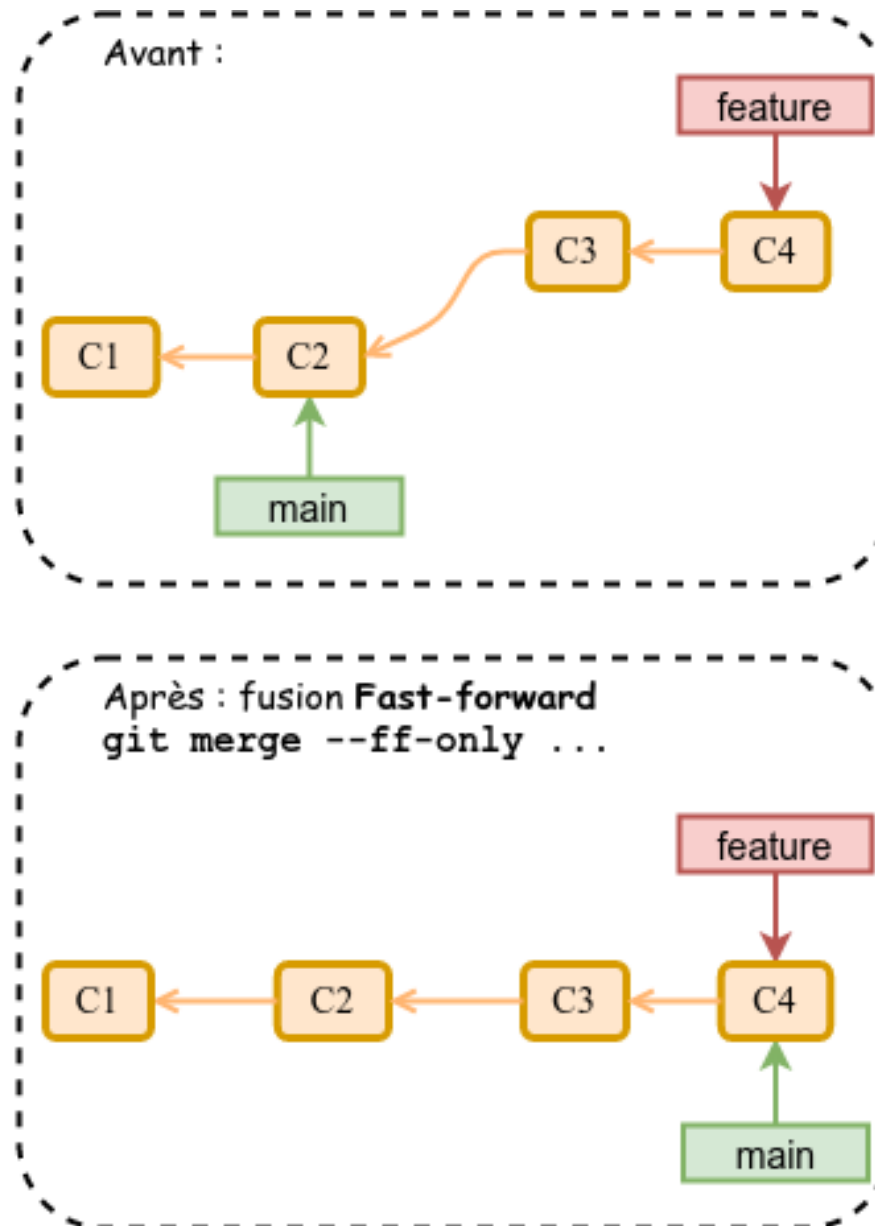


Stratégies de fusion

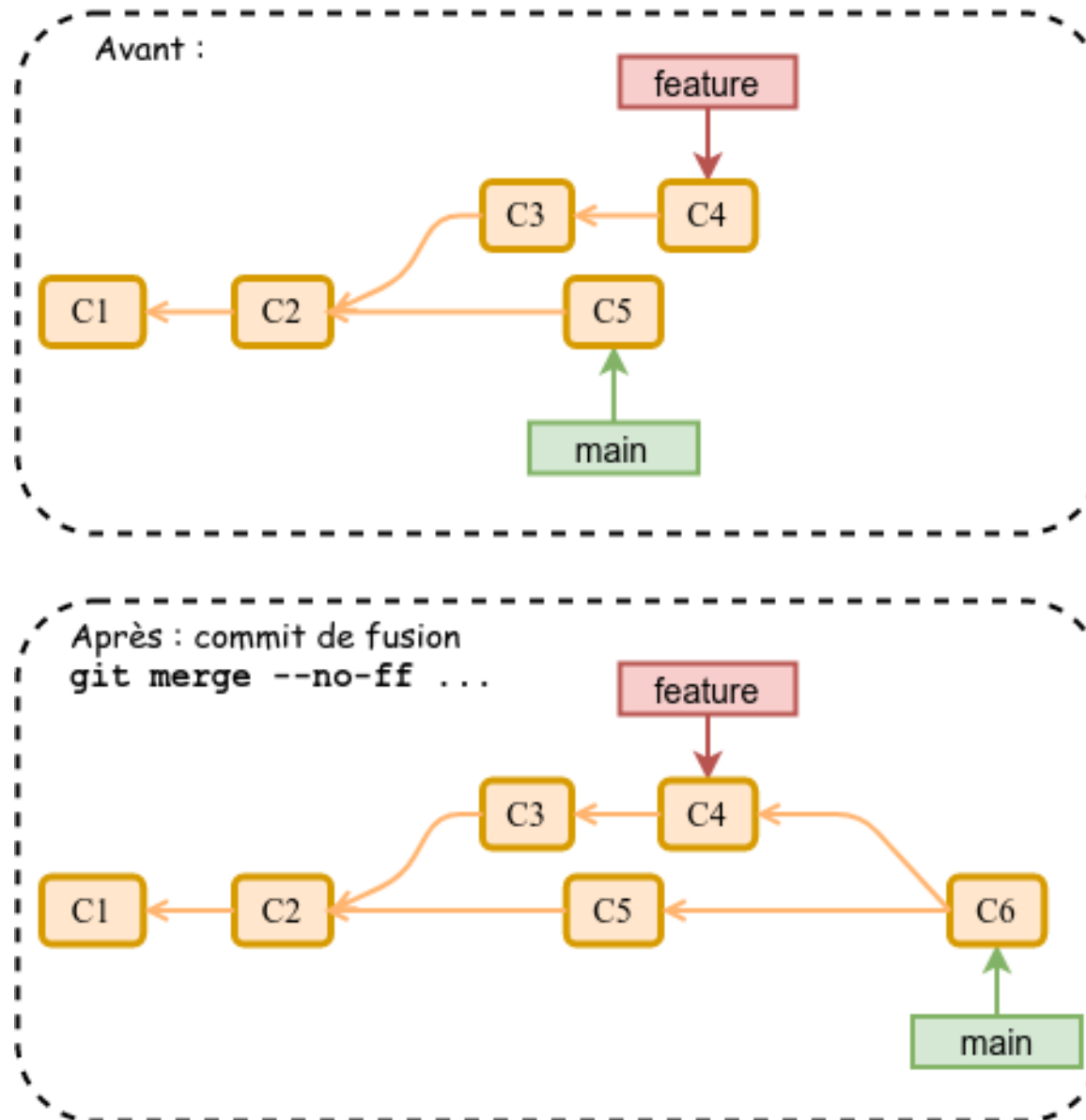
- **Avance rapide (*Fast Forward*)** : c'est la fusion utilisée **par défaut** par `git merge` si c'est possible. Git déplace (vers l'avant) les *commits* de la branche *feature* vers la branche destination *main* si il n'y a pas eu de nouveaux *commits* sur cette branche. On peut réaliser cette fusion avec l'option `--ff-only`.
- **Commit de fusion (*merge commit*)** : lorsque l'historique de développement a divergé, `git merge` réalise une fusion à trois sources (*three-way merge*) en utilisant les deux *commits* au sommet des deux branches (C4 et C5) ainsi que leur plus proche ancêtre commun (C2) pour créer un nouveau *commit* (C6). On peut réaliser cette fusion avec l'option `--no-ff`.
- **Squash** : on obtient un nouveau *commit* qui regroupe tous les *commits* de la branche. Pour réaliser cette fusion, il faut ajouter l'option `--squash`.



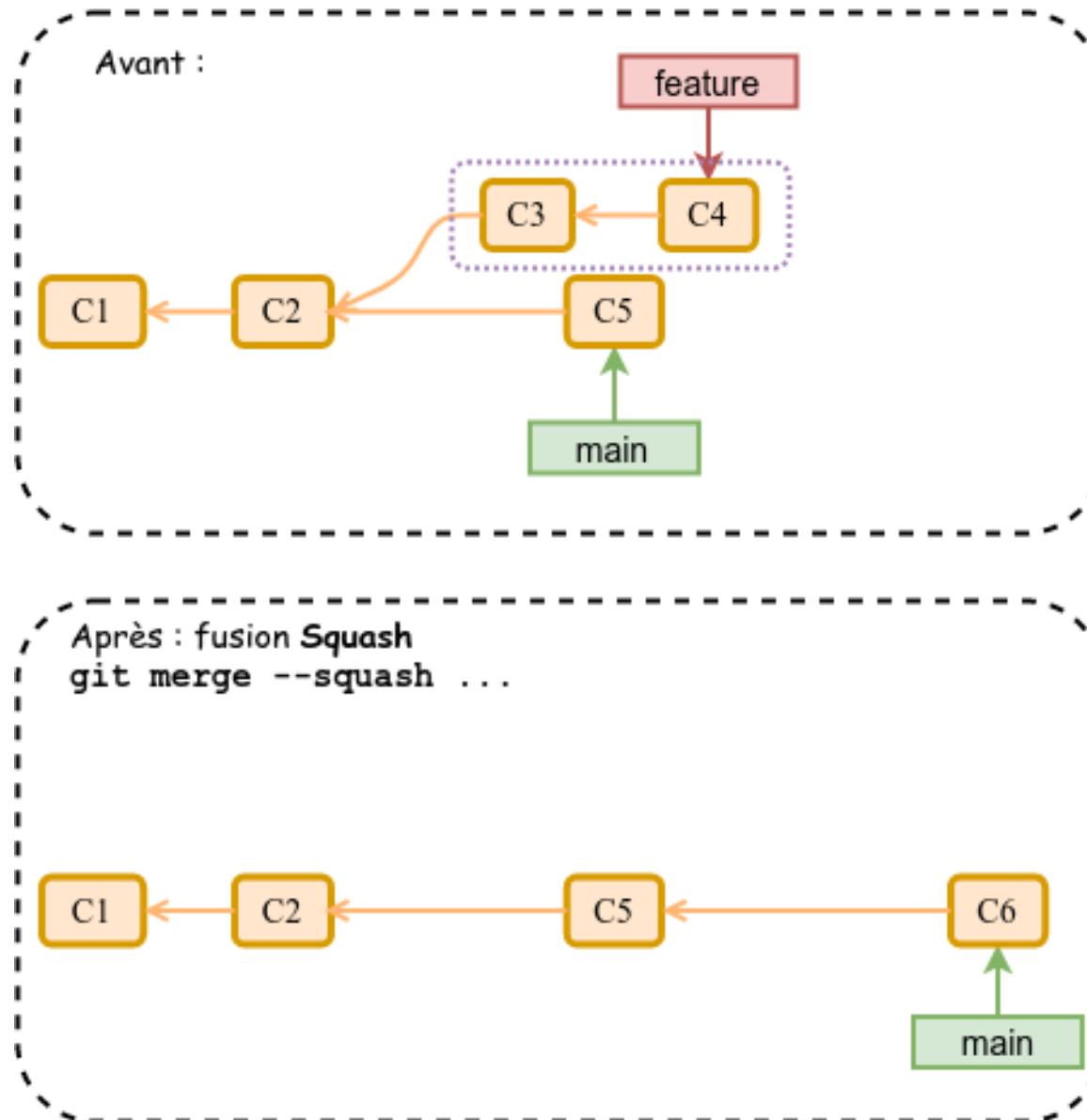
Avance rapide (*Fast Forward*)



Commit de fusion



Squash



Conflit de fusion

- Il est possible qu'une fusion (*merge*) ne puisse pas être réalisée automatiquement par Git.
- Cela arrive lorsqu'une même partie d'un fichier a été modifiée dans deux branches distinctes.
- Lorsque Git rencontre un conflit au cours d'une fusion, il l'indique dans les fichiers concernés avec des **délimiteurs** (<<<<<<<, ===== et >>>>>>>) qui marquent les deux côtés du conflit.
- Pour résoudre le conflit, il faut choisir une partie ou l'autre ou bien fusionner les deux contenus "à la main".
- On peut ensuite terminer la fusion (suivre les indications de git status) en faisant un git commit.



workflow git

- Un **workflow Git** est une méthode, un processus de travail, une recette ou une recommandation sur la façon d'utiliser Git pour accomplir un travail de manière cohérente et productive.
- Il n'existe pas de processus standardisé sur la façon d'interagir avec Git.
- Il est important de s'assurer que l'équipe de projet est d'accord sur la façon dont le flux de modifications sera appliqué. Un *workflow* Git doit donc être défini.
- Il existe plusieurs *workflow* Git connus qui peuvent être utilisés : *workflow* centralisé, *workflow* de branche de fonctionnalité, *workflow Gitflow*, etc.

www.atlassian.com/fr/git/tutorials/comparing-workflows



Gitflow

- Le *workflow* **Gitflow** définit un modèle de branchement strict conçu autour de la version du projet.
- Ce *workflow* n'ajoute pas de nouveaux concepts ou commandes.
- **Gitflow** permet de gérer les bugs (*issues*), les nouvelles fonctionnalités (*features*) et les versions (*releases*) en attribuant des rôles très spécifiques à différentes branches et définit comment et quand elles doivent interagir.

Il existe des extensions `git-flow` à Git pour intégrer le *workflow* Gitflow.

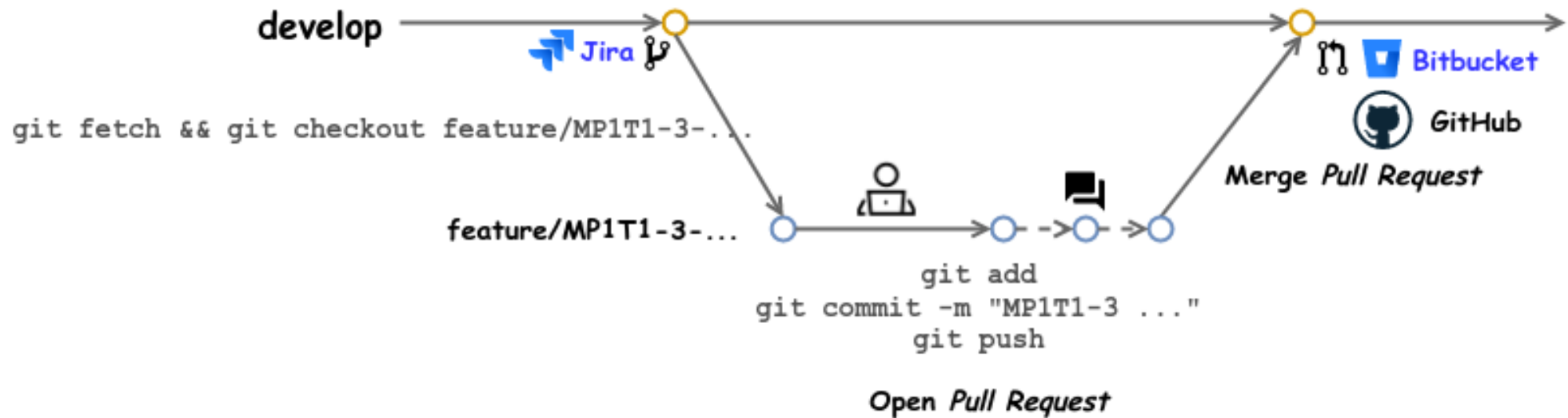
Modèle de branches



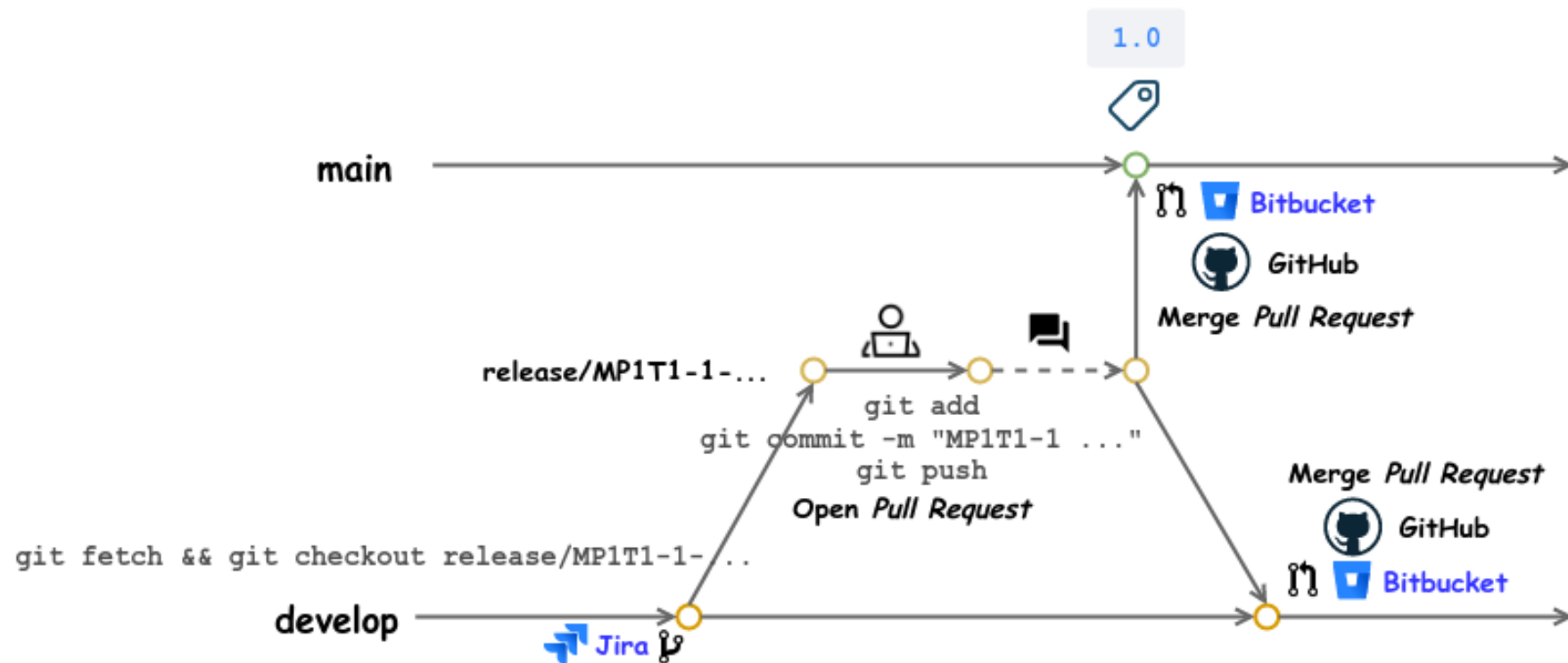
Rôles des branches

- pour les **branches permanentes** :
 - La **branche *master*** stocke l'historique des versions officielles. Tous les *commits* de cette branche sont étiquetés avec un numéro de version (*tags*).
 - La **branche *develop*** est créée à partir de la branche *master*. Elle sert de branche d'intégration pour les fonctionnalités. Cette branche contiendra l'historique complet du projet.
- pour les **branches temporaires** :
 - Les **branches *features-xxxx*** permettent de travailler sur des nouvelles fonctionnalités. Elles sont créées directement à partir de la branche *develop* et une fois le travail fini, fusionnées vers la branche *develop*.
 - Les **branches *release-xxxx*** permettent de travailler sur une livraison (généralement des tâches dédiées à la documentation). On les crée à partir de *develop* puis on les fusionne dans *master* en leur attribuant un numéro de version (*tag*).
 - Les **branches *hotfix-xxxx*** permettent de publier rapidement (*hot*) une correction (*fix*) depuis la branche *master*. Ces branches seront ensuite fusionnées vers la branche *master* et *develop*.

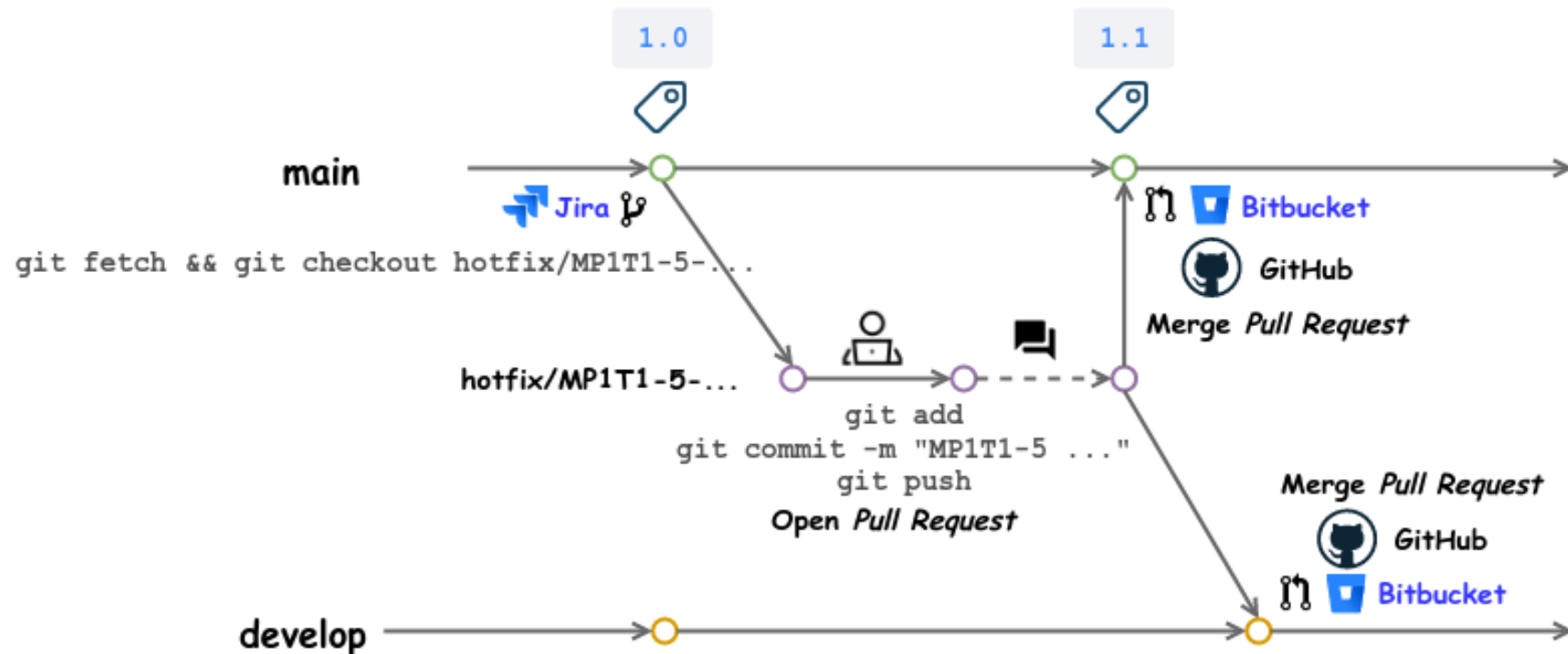
Réalisation d'une fonctionnalité



Réalisation d'une *release*



Correction d'un *bug*



Projets BTS SN

- Les branches (*feature*, *release* et *hotfix*) seront créées dans **Jira à partir d'un ticket**.
- Les fusions seront réalisées lors d'une revue de code en utilisant les ***Pull Requests* dans GitHub ou Bitbucket**.

Lorsqu'elle désigne un travail bien identifié du projet (une fonctionnalité, une *release* ou un correctif), il est préférable (obligatoire) que cela reste visible dans le graphe d'historique, même lorsque la branche est supprimée. Pour éviter que Git utilise par défaut une avance rapide (*Fast Forward*) si c'est possible, il faudra réaliser un *commit* de fusion avec l'option `--no-ff`.

Jira / GitHub

📢 Donnez votre avis 1 🍏 🔄 ⋮ ✕

← GitHub



Create Branches PRs Tags

Repository*

btssn-lasalle84/mp1-team0

Base branch*

🔗 develop

Branch name*

feature/MP1T0-2-initialisation-du-projet

⚙ Branch Name Template

Create branch

← GitHub



Branch **feature/MP1T0-2-initialisation-du-projet** has been successfully created.

Use this command to check out your branch:

git fetch && git checkout feature/MP1T0-2-initialis



Jira / Bitbucket

Create branch

Repository

btssn-avignon/miniprojet1-team1

Type ⓘ

Feature

From branch

develop

Branch name

feature/ MP1T1-2-initialisation-du-projet



develop

feature/MP1T1-2-initialisation-du-proj...

Create

Cancel

btssn-avignon / miniprojet1-team1 / miniprojet1-team1 / Branches

feature/MP1T1-2-initialisation-du-projet

develop [changer la destination](#)

feature/MP1T1-2-initialisation-du-proj...

Consultez cette branche

Cette branche ne contient aucune modification pour l'instant. Consultez-la depuis votre ordinateur local pour la modifier.

[Consulter via Sourcetree](#)

You can also use this command to check out your branch:

```
git fetch && git checkout feature/MP1T1-2-initialisation-du-proj...
```

Choisir *commit* de fusion (*Pull Request*)

Bitbucket

Merge strategy

Merge commit ▼

Merge commit
git merge --no-ff

Squash
git merge --squash

Fast forward
git merge --ff-only

GitHub

Merge pull request ▼

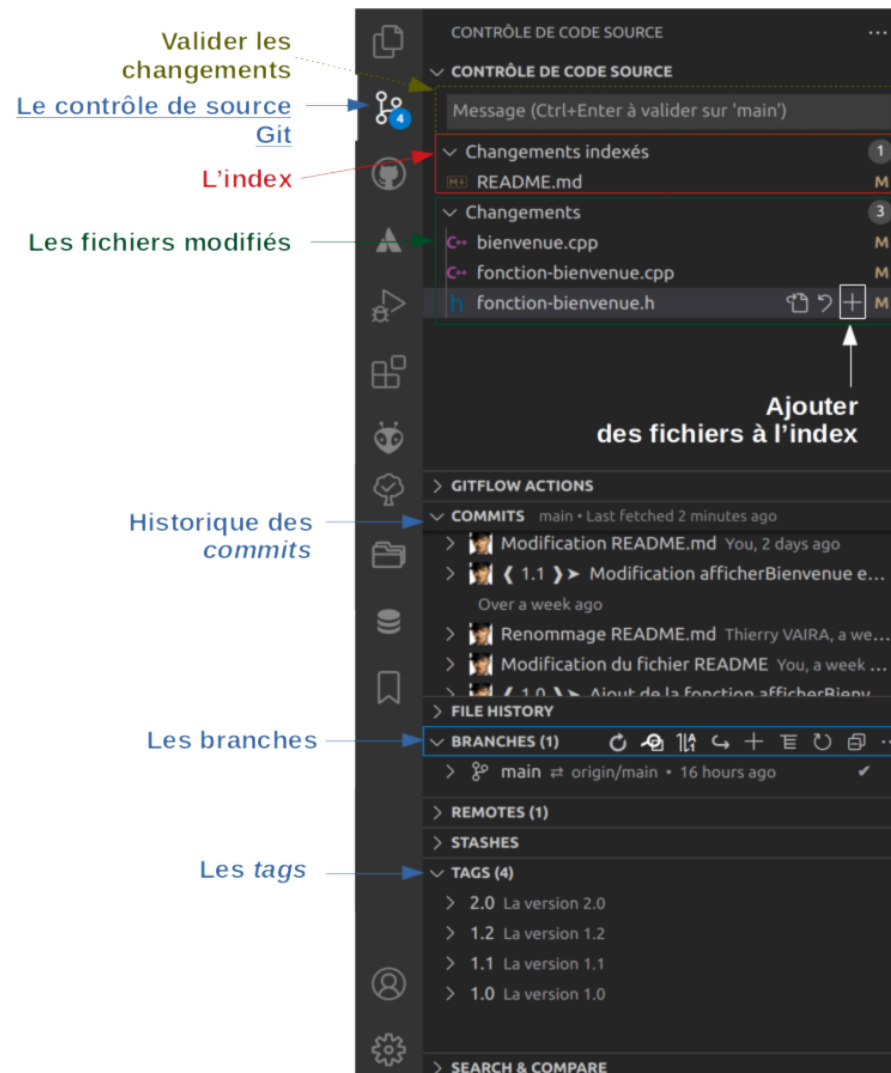
✓ **Create a merge commit**
All commits from this branch will be added to the base branch via a merge commit.

Squash and merge
The 1 commit from this branch will be added to the base branch.

Rebase and merge
The 1 commit from this branch will be rebased and added to the base branch.

Environnement de développement intégré (EDI ou IDE)

La plupart des environnements de développement intègre Git ou propose des extensions pour le faire. Le contrôle de source (SCM) dans VS Code :



Les outils graphiques

Il existe de nombreuses interfaces graphiques permettant de gérer des projets Git :

- une interface web avec GitWeb
- une interface de visualisation détaillée et graphique avec gitk

Il existe également de nombreuses autres applications :

- qgit, Giggie, GitExtensions, TortoiseGit, SourceTree, GitEye, ...

GitEye

CollabNet GitEye

File Edit Search Git Window Help

Git Repositories CollabNet Sites

team-0 [master] - /home/iris/Documents/jira/team-0/.git

- Branches
 - Local
 - origin/develop 90cd7a7 Fusion effectuée feature/T0-2-coder-fonc-2 (pull request #1)
 - origin/feature/T0-1-coder-fonc-1 0dd01b3 réalisation de la fonc 1
 - origin/feature/T0-2-coder-fonc-2 e504848 Ajout du fichier projet.txt
 - origin/master 90cd7a7 Fusion effectuée feature/T0-2-coder-fonc-2 (pull request #1)
 - origin/release/1.0 90cd7a7 Fusion effectuée feature/T0-2-coder-fonc-2 (pull request #1)
 - Remote Tracking
 - origin/develop 90cd7a7 Fusion effectuée feature/T0-2-coder-fonc-2 (pull request #1)
 - origin/feature/T0-1-coder-fonc-1 0dd01b3 réalisation de la fonc 1
 - origin/feature/T0-2-coder-fonc-2 e504848 Ajout du fichier projet.txt
 - origin/master 90cd7a7 Fusion effectuée feature/T0-2-coder-fonc-2 (pull request #1)
 - origin/release/1.0 90cd7a7 Fusion effectuée feature/T0-2-coder-fonc-2 (pull request #1)
- Tags
- References
- Remotes
 - origin
 - https://tvaira@bitbucket.org/btssn-avignon/team-0.git
 - https://tvaira@bitbucket.org/btssn-avignon/team-0.git
- Working Tree - /home/iris/Documents/jira/team-0 [team-0 master]
 - .git
 - .gitignore
 - .project
 - projet.txt

1 item selected

Dashboard Git Files History Task List Builds

Repository: team-0

Id	Message	Author	Authored Date	Committer	Committed Date
90cd7a7	1.0 develop master release/1.0 origin/HEAD origin/develop origin/master origin/release/1.0 HEAD	Vaira Thierry	12 hours ago	Thierry Vaira	12 hours ago
f2ea333	réalisation de la fonc 2	Vaira Thierry	12 hours ago	Vaira Thierry	12 hours ago
4f8e9e8	Merged develop into feature/T0-2-coder-fonc-2	Thierry Vaira	13 hours ago	Thierry Vaira	13 hours ago
9f5d94c	Revert "réalisation de la fonc 2"	tvaire	13 hours ago	tvaire	13 hours ago
9e5ff5e	réalisation de la fonc 2	tvaire	2 days ago	tvaire	2 days ago
0dd01b3	origin/feature/... réalisation de la fonc 1	tvaire	2 days ago	tvaire	2 days ago
e504848	origin/feature/... Ajout du fichier projet.txt	vaira	3 days ago	vaira	3 days ago
ff8740	Initial commit	Thierry Vaira	3 days ago	Thierry Vaira	3 days ago

commit 90cd7a745004cf06db097aa7e971d96306063740
 Author: Vaira Thierry <rk42nftjrm@privaterelay.appleid.com> 2021-07-09 21:03:52
 Committer: Thierry Vaira <vaira@lasalle84.org> 2021-07-09 21:03:52
 Parent: 0dd01b30c83b0e819150193cc49e8cdd79fba232 (réalisation de la fonc 1)
 Parent: f2ea33352ac5786cbb5f9823116a010dd432ef09 (réalisation de la fonc 2)
 Branches: develop, master, release/1.0, origin/HEAD, origin/develop, origin/master, origin/release/1.0
 Tags: 1.0

Fusion effectuée feature/T0-2-coder-fonc-2 (pull request #1)

réalisation de la fonc 2

Approuvé par : Thierry Vaira



Les principales commandes I

Git est un ensemble de commandes indépendantes dont les principales sont :

- `git init` crée un nouveau dépôt ;
- `git clone` clone un dépôt distant ;
- `git add` ajoute le contenu du répertoire de travail dans la zone d'index pour le prochain *commit* ;
- `git status` montre les différents états des fichiers du répertoire de travail et de l'index ;
- `git diff` montre les différences ;
- `git commit` enregistre dans la base de données (le dépôt) un nouvel instantané avec le contenu des fichiers qui ont été indexés puis fait pointer la branche courante dessus ;
- `git branch` liste les branches ou crée une nouvelle branche ;



Les principales commandes II

- `git checkout` permet de basculer de branche et d'en extraire le contenu dans le répertoire de travail ;
- `git merge` fusionne une branche dans une autre ;
- `git log` affiche la liste des *commits* effectués sur une branche ;
- `git fetch` récupère toutes les informations du dépôt distant et les stocke dans le dépôt local ;
- `git push` publie les nouvelles révisions sur le dépôt distant ;
- `git pull` récupère les dernières modifications distantes du projet et les fusionne dans la branche courante ;
- `git tag` liste ou crée des *tags* ;
- `git stash` stocke de côté un état non commité afin d'effectuer d'autres tâches.

